Predictive Maintenance for NASA C-MAPSS: An End-to-End PHM System with Calibrated RUL Prediction, Risk Logic, and Scenario Analysis

Author 1
Artificial Intelligence and Advanced Large Models Course
Beihang University, Beijing, China

***Abstract*—Remaining Useful Life (RUL) estimation is a core task in prognostics and health management (PHM), yet many studies on the NASA C-MAPSS benchmark stop at predictive accuracy and do not address how model outputs are converted into deployable, auditable maintenance decisions. This paper presents an end-to-end PHM system built on NASA C-MAPSS that integrates three explicitly separated layers: a predictive layer for RUL estimation with calibrated confidence bands, a deterministic agent layer for risk scoring and maintenance recommendation, and a dashboard layer for operator-facing analysis, scenario exploration, and technical auditability. The predictive layer evaluates four candidate models—Random Forest, HistGradientBoosting, LSTM, and GRU—across the four C-MAPSS subsets under both predictive and engineering-oriented criteria. Although GRU achieves the best raw global RMSE, the deployed champion is Random Forest because it provides stronger stability, clearer integration boundaries, and more reliable serving behavior, with HistGradientBoosting retained as fallback. The system further operationalizes prediction through contract-driven interfaces, degraded-state handling, scenario comparison, and traceable audit records exposed in interactive dashboard views. The main result is not only a benchmarked RUL study, but a reproducible PHM application that demonstrates how calibrated prediction, deterministic decision logic, and audit-oriented system design can transform a benchmark workflow into a deployment-oriented engineering artifact.**

***Index Terms*—predictive maintenance, prognostics and health management, NASA C-MAPSS, remaining useful life, PHM systems, traceability, scenario analysis**

I. Introduction

Prognostics and Health Management (PHM) has become a central research and engineering area for safety-critical systems in which failures are costly, disruptive, or operationally unacceptable. Within PHM, Remaining Useful Life (RUL) estimation is one of the most widely studied tasks because it provides a compact prediction of how long a system can continue operating before reaching a failure threshold. In the aerospace domain, reliable RUL estimation is especially important because maintenance decisions must balance safety, availability, and operational cost under uncertainty. For this reason, the NASA C-MAPSS benchmark has become one of the standard datasets for evaluating data-driven RUL models on simulated turbofan degradation trajectories [1], [2].

However, a persistent gap remains between benchmark-oriented RUL studies and deployable PHM systems. A large portion of the literature on C-MAPSS focuses primarily on predictive performance, often comparing regression models through metrics such as RMSE or MAE and emphasizing whether a given architecture outperforms another on one or more benchmark subsets. Sequence models such as LSTM- and GRU-based approaches are especially prominent in this line of work because of their ability to capture temporal degradation patterns [3]. Yet a real maintenance-support system requires more than a point estimate. It must also communicate uncertainty, translate predictive outputs into actionable risk levels and recommendations, support scenario analysis, preserve traceability across components, and remain robust under degraded operating conditions. These requirements are central in practical PHM deployment and in broader maintenance analytics literature, but they are often weakly represented in benchmark-only studies [4], [5], [6].

This paper addresses that gap by designing and evaluating an end-to-end PHM system over NASA C-MAPSS rather than treating RUL prediction as an isolated regression problem. The proposed system is structured around three explicitly separated layers. First, a predictive layer estimates RUL and augments the point prediction with calibrated confidence information. Second, a deterministic agent layer converts predictive outputs into auditable risk scores, risk levels, and maintenance recommendations through explicit rule-based logic. Third, a dashboard layer operationalizes these outputs through summary, analysis, scenario, and technical-audit workflows. In this way, the contribution of the work is not limited to selecting a strong predictive model; it lies in showing how benchmark-based prediction can be embedded into a system architecture that is interpretable, traceable, and deployment-oriented.

A second central idea of this work is that the best benchmark model is not necessarily the best deployed model. The implemented predictive layer evaluates four candidate models—Random Forest (RF), HistGradientBoosting (HGB), LSTM, and GRU—across the four C-MAPSS subsets. Although GRU achieves the best raw global RMSE in the current evaluation, the deployed champion is RF because it provides stronger operational stability, simpler integration, clearer serving behavior, and a more reliable deterministic boundary with the rest of the system, while HGB is retained as fallback :contentReference[oaicite:2]index=2. This design decision reflects a broader engineering principle: model selection in PHM should be driven not only by raw predictive accuracy, but also

by integration readiness, robustness, traceability, and system-level reliability.

The main contributions of this work are as follows:

1) An end-to-end PHM architecture for NASA C-MAPSS that integrates calibrated RUL prediction, deterministic risk logic, scenario analysis, and dashboard-based technical auditability.
2) A comparative predictive layer including RF, HGB, LSTM, and GRU, with champion selection based on both predictive metrics and deployment-oriented engineering criteria.
3) A deterministic agent layer that transforms model outputs into traceable maintenance decisions through explicit risk scoring, recommendation rules, validation mechanisms, and degraded-state handling.
4) A dashboard layer that exposes the full workflow through operator-facing summary and analysis views, what-if scenario exploration, and technical-audit interfaces linked to service state and trace records.
5) A reproducible system artifact showing how a benchmark RUL workflow can be extended into a deployment-oriented PHM application rather than remaining a standalone modeling exercise.

The remainder of the paper is organized as follows. Section II reviews the relevant literature on C-MAPSS modeling, PHM decision support, and trustworthy engineering AI systems. Section III presents the system and problem background, including the NASA C-MAPSS setting and the role of RUL within the proposed PHM stack. Section IV describes the methodology and overall system architecture. Section V details the experimental setup. Section VI reports predictive, engineering, and dashboard-level results. Section VII discusses the main implications of the findings, particularly the distinction between benchmark winners and deployment champions. Section VIII states the main limitations of the current system. Finally, Section IX concludes the paper.

II. Literature Review

A. NASA C-MAPSS and data-driven RUL estimation

The NASA C-MAPSS benchmark has become one of the most widely used reference datasets for data-driven prognostics research on aircraft engines. Its importance stems from two complementary factors: first, it provides multivariate run-to-failure trajectories generated from a realistic turbofan simulation environment; second, it addresses a longstanding limitation in PHM research, namely the scarcity of openly available run-to-failure datasets suitable for rigorous method comparison [1], [2]. In that sense, C-MAPSS helped standardize evaluation practice in a field where direct access to fleet-scale degradation data is often restricted by cost, instrumentation limitations, or proprietary constraints.

A substantial body of work has used C-MAPSS primarily as a supervised regression benchmark for Remaining Useful Life (RUL) estimation. In this literature, predictive quality is typically assessed through aggregate error metrics such as RMSE or MAE, with comparisons made across benchmark subsets and model families. This model-centric view is consistent with broader data-driven prognostics research, where statistical and machine learning methods have long been surveyed as central tools for RUL estimation under degradation uncertainty [7]. Within the C-MAPSS context, early neural approaches already showed that sequential models could exploit degradation histories more effectively than static formulations. Heimes, for example, applied recurrent neural networks to the 2008 PHM challenge and demonstrated that temporal architectures could provide competitive RUL estimates from run histories [8].

As deep learning matured, the C-MAPSS literature expanded beyond recurrent models. Convolutional and hybrid deep architectures were introduced to learn degradation representations directly from multivariate sensor streams, reducing dependence on hand-crafted features. Babu et al. proposed a deep convolutional regression framework for RUL estimation, showing that convolutional filters could extract useful degradation patterns from sequential sensor measurements [9]. Later work extended this direction through deeper and more structured temporal pipelines. Ellefsen et al. combined semi-supervised representation learning with deep architectures for turbofan degradation, highlighting the value of feature learning and pretraining under complex operating conditions and fault modes [10]. Together with LSTM-based formulations such as Zheng et al. [3], these works established a dominant paradigm in which C-MAPSS is primarily treated as a benchmark for optimizing predictive architectures.

B. From point prediction to uncertainty-aware prognostics

Despite the dominance of model-comparison studies, PHM research has repeatedly emphasized that prognostics is not only about producing a point estimate. Ramasso and Saxena argue that prognostic methodologies must be understood along a continuum from statistical and machine learning techniques to explicit uncertainty management, because practical maintenance decisions are inherently made under incomplete information [4]. Similarly, Sikorska et al. show that industrial RUL estimation methods must be judged not only by nominal accuracy, but also by interpretability, applicability, and operational robustness [5]. In other words, a predictor is useful only insofar as its outputs can be trusted, interpreted, and acted upon.

This distinction is especially important because prognostic error is not operationally neutral. In many engineering contexts, an early warning and a late warning do not carry symmetric consequences, and aggregate regression metrics alone may fail to reflect this asymmetry. Saxena et al. explicitly addressed this issue by discussing prognostic performance metrics that distinguish between early and late prediction behavior and by framing evaluation as a decision-relevant problem rather than a purely statistical one [11]. This perspective is highly relevant to RUL systems built on C-MAPSS: a model that performs well in aggregate error may still be inadequate if it cannot express confidence, does not behave consistently across subsets, or cannot be interpreted in the context of maintenance decisions.

C. PHM decision support beyond standalone prediction

A second body of literature, closer to industrial analytics and deployment, argues that predictive models create value only when embedded in a broader decision-support workflow. Lee et al. describe smart analytics in Industry 4.0 as part of an integrated service architecture in which data, models, and decisions must be connected to operational action [6]. From this viewpoint, the output of a prognostic model is not the end product; rather, it is an intermediate artifact that must be translated into risk interpretation, intervention logic, scheduling implications, and traceable system behavior.

However, this system-oriented perspective is still only partially represented in much of the benchmark-driven C-MAPSS literature. Many studies stop at reporting RUL predictions and do not explicitly formalize how predictive outputs are transformed into maintenance states, risk levels, fallback behavior, or auditable decision records. Likewise, scenario exploration, service-status handling, and dashboard-mediated technical inspection are rarely treated as first-class research objects, even though these capabilities are essential for engineering deployment in safety-critical settings. As a result, the literature remains uneven: benchmark sophistication has advanced rapidly, but the integration of prediction into reproducible PHM decision systems remains limited.

D. Research gap

Taken together, the literature suggests four conclusions. First, C-MAPSS is a mature and well-established benchmark for comparing data-driven RUL models under controlled turbofan degradation scenarios [1], [2]. Second, the benchmark literature has evolved from classical statistical and recurrent approaches toward deeper sequential and representation-learning models, including RNN-, CNN-, LSTM-, and semi-supervised architectures [8], [9], [3], [10]. Third, PHM research has consistently stressed that useful prognostics must address uncertainty, interpretability, and evaluation beyond raw regression error [4], [5], [11]. Fourth, industrial analytics perspectives emphasize that predictive intelligence becomes valuable only when embedded into operational decision support [6].

What remains insufficiently addressed is the integration of these strands into a unified, deployment-oriented PHM artifact built on top of a benchmark such as C-MAPSS. In particular, there is still limited work that explicitly connects calibrated RUL estimation, deterministic risk translation, scenario analysis, dashboard-mediated inspection, and technical auditability within a single reproducible architecture. This paper addresses that gap by treating C-MAPSS not merely as a predictive benchmark, but as the basis for an end-to-end PHM system in which model outputs are converted into traceable maintenance intelligence.

III. System and Problem Background

A. NASA C-MAPSS as a turbofan prognostics benchmark

The Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) is a widely used benchmark for data-driven prognostics of aircraft engines. It was developed to provide realistic multivariate run-to-failure trajectories in a context where publicly available real-world degradation datasets are limited [1], [2]. The simulator represents a large commercial turbofan engine operating under varying flight and control conditions, and produces time series of operational settings and sensor measurements that evolve as engine health deteriorates.

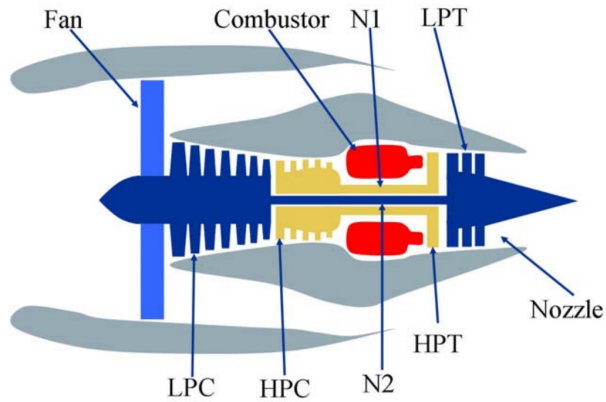


Fig. 1. Conceptual turbofan engine context for the C-MAPSS benchmark, showing the main modules typically referenced in prognostics studies: fan, LPC, HPC, combustor, HPT, LPT, and nozzle.

A key advantage of C-MAPSS is that it bridges physical structure and data-driven evaluation. The benchmark does not consist of arbitrary synthetic sequences; rather, it is generated from an aero-propulsion simulation framework with module-level degradation behavior and operational variability. As a result, it has become a standard testbed for Remaining Useful Life (RUL) estimation methods, especially in studies where the goal is to infer degradation trends from multivariate sensor histories [1], [2].

B. Turbofan engine context

From a system perspective, the benchmark corresponds to a turbofan engine composed of physically distinct modules such as the fan, the low-pressure compressor (LPC), the high-pressure compressor (HPC), the combustor, the high-pressure turbine (HPT), the low-pressure turbine (LPT), and the nozzle. These modules interact thermodynamically and mechanically, and degradation in one or more of them affects the observed operational and sensor variables over time. In the original challenge setting, fault propagation was modeled through changes in flow and efficiency variables at the module level, and these internal degradations were reflected indirectly in the sensor streams made available to participants [2].

Although C-MAPSS exposes only operational settings and sensor outputs to the learning pipeline, understanding the engine structure remains important for interpretation. The benchmark is therefore not simply a generic multivariate time-series problem; it is a structured prognostics problem in which the observed trajectories are tied to the degradation behavior of an aero-engine system. This physical grounding is one reason why C-MAPSS remains a useful benchmark not only

TABLE I
C-MAPSS SUBSETS AND PROBLEM STRUCTURE.

Subset	Train units	Test units	Conditions	Fault modes
FD001	100	100	1	1
FD002	260	259	6	1
FD003	100	100	1	2
FD004	248	249	6	2

for predictive modeling, but also for PHM-oriented system design.

C. Problem formulation and benchmark subsets

Each C-MAPSS record corresponds to a given engine unit observed at a specific operational cycle. The standard representation includes a unit identifier, a cycle index, three operational setting variables, and multiple sensor measurements. The prognostics task is to estimate the Remaining Useful Life of each unit from its degradation history. In practical terms, the model receives the partial trajectory of a unit up to cycle t and must estimate how many cycles remain before failure.

The benchmark is divided into four standard subsets, denoted FD001 through FD004. These subsets differ in the number of operating conditions and fault modes represented. FD001 and FD003 involve a single operating condition, whereas FD002 and FD004 involve multiple operating conditions. Likewise, FD001 and FD002 involve a single fault mode, while FD003 and FD004 involve two fault modes. This structure makes the benchmark progressively more challenging as one moves from simpler to more heterogeneous degradation scenarios. Table I summarizes this organization.

This subset design is important for the present work because it highlights a distinction between predictive difficulty and deployment difficulty. A model may perform well globally while still exhibiting unstable or uneven behavior across subsets, operating regimes, or fault structures. Since the objective of this paper is not limited to achieving the lowest aggregate error, but to building a deployable PHM system, subset-level behavior is treated as part of the engineering background rather than as a secondary reporting detail.

D. From benchmark RUL estimation to PHM system context

In the classical benchmark formulation, the output of interest is a scalar RUL estimate. In the present work, that scalar prediction is treated as only the first stage of a larger PHM workflow. The benchmark therefore serves as the predictive foundation of a broader system architecture composed of three layers: a predictive layer, a deterministic agent layer, and a dashboard layer.

Within this architecture, the predictive layer is responsible for transforming the C-MAPSS trajectory representation into a calibrated RUL estimate and an associated confidence band. The agent layer then interprets those outputs through deterministic risk logic, generating risk scores, categorical maintenance states, and recommendation text. Finally, the dashboard layer presents this information through operator-facing and audit-facing views, including summary panels, scenario comparison, and technical trace inspection. In this sense, the benchmark target is preserved, but its role changes: RUL is no longer the final product of the system, but an intermediate quantity that supports maintenance intelligence and auditable decision support.

This shift in perspective is central to the paper. C-MAPSS provides the predictive substrate, but the contribution of the proposed work lies in showing how that substrate can be embedded into a complete PHM application. Accordingly, the remainder of the paper treats the benchmark not only as a modeling dataset, but as the technical basis for calibrated prediction, deterministic risk translation, scenario reasoning, and traceable engineering deployment.

IV. METHODOLOGY

A. Overview of the end-to-end PHM architecture

The proposed methodology is organized as a three-layer PHM architecture composed of a predictive layer, a deterministic agent layer, and a dashboard layer. The design objective is to move from benchmark-oriented Remaining Useful Life (RUL) estimation to a deployment-oriented system in which predictive outputs are converted into traceable, inspectable, and actionable maintenance intelligence.

At a high level, the workflow is as follows. First, the predictive layer receives the operational and sensor state of an engine unit and produces a point estimate of RUL together with calibrated confidence information. Second, the agent layer consumes these outputs and applies deterministic logic to derive a risk score, a categorical risk level, a recommendation text, and audit-related state information. Third, the dashboard layer presents the resulting information through user-facing and audit-facing views, while also supporting scenario exploration and contract-level inspection. This separation is intentional: the predictive component is responsible for estimation, the agent component for decision translation, and the dashboard component for operational interpretation.

B. Predictive layer

1) Preprocessing and feature policy: The predictive layer is built on top of the standard C-MAPSS representation, where each observation corresponds to a unit-cycle state described by operational settings and sensor measurements. In the deployed pipeline, preprocessing follows three main principles: (i) removal of low-variance variables, (ii) training-only scaling to prevent information leakage, and (iii) preservation of explicit Remaining Useful Life policies for both raw and capped RUL formulations.

A subset of variables is excluded because of low variance or weak utility in the adopted predictive pipeline. The retained feature set consists of 16 variables (Table II).

This feature policy serves two purposes. First, it simplifies the predictive interface and reduces redundancy in the row-based learning pipelines. Second, it defines a stable contract for downstream inference and scenario handling, since all

TABLE II
SELECTED AND REMOVED VARIABLES IN THE DEPLOYED PREDICTIVE PIPELINE.

Category	Variables
Selected predictive features (16)	`op_setting_1`, `op_setting_2`, `sensor_2`, `sensor_3`, `sensor_4`, `sensor_7`, `sensor_8`, `sensor_9`, `sensor_11`, `sensor_12`, `sensor_13`, `sensor_14`, `sensor_15`, `sensor_17`, `sensor_20`, `sensor_21`
Removed low-variance features (8)	`op_setting_3`, `sensor_1`, `sensor_5`, `sensor_6`, `sensor_10`, `sensor_16`, `sensor_18`, `sensor_19`

deployed components depend on a known feature subset and explicit validation rules.

2) Candidate model portfolio: The predictive layer evaluates four candidate models selected to cover both tabular and sequential formulations of the RUL task:

- **Random Forest Regressor (RF):** a tree-based ensemble used as a robust tabular baseline and champion candidate.
- **HistGradientBoosting Regressor (HGB):** a boosting-based tabular alternative used as both baseline and fallback candidate.
- **Long Short-Term Memory (LSTM):** a sequence model designed to capture temporal degradation patterns through windowed trajectories.
- **Gated Recurrent Unit (GRU):** a recurrent sequence model used as a strong temporal challenger.

The motivation for including both tabular and sequential models is methodological rather than merely competitive. Tabular models offer simplicity, fast iteration, and stable serving behavior, whereas recurrent models are designed to exploit temporal degradation structure more directly. The comparison therefore reflects a broader design question: whether the incremental predictive gains from sequence-aware architectures justify their added integration and deployment complexity within a full PHM stack.

3) Training and evaluation perspective: Model comparison is not driven exclusively by aggregate predictive accuracy. Instead, the predictive layer is evaluated under a mixed criterion combining predictive and engineering-oriented considerations. The predictive side includes global and subset-level error behavior, while the engineering side includes serving simplicity, stability, integration readiness, inference consistency, and compatibility with downstream deterministic decision logic.

This distinction is essential in the present methodology. A model that obtains the lowest global RMSE is not automatically selected for deployment if its integration cost, service behavior, or interpretability creates avoidable system-level fragility. Accordingly, the champion model is selected not only from the standpoint of benchmark performance, but also from the standpoint of system operability.

4) Champion selection and fallback design: The evaluated portfolio yields a best raw global RMSE for the GRU model, but the deployed champion is Random Forest. HistGradient-Boosting is retained as the deterministic fallback model. This champion decision reflects the system philosophy of the work: deployment quality is defined not only by predictive sharpness, but also by architectural stability, clearer inference boundaries, simpler serving behavior, and lower integration overhead with the rest of the PHM workflow.

The fallback mechanism is an explicit part of the methodology rather than a post hoc implementation detail. If the champion model becomes unavailable or fails validation, the system can propagate state information and continue operating through a controlled fallback path. This design improves robustness and allows the downstream agent layer to retain deterministic behavior even when the predictive layer is partially degraded.

5) Inference and confidence calibration: At inference time, the predictive layer receives a validated engine-state payload and returns at least four key elements:

1) a point estimate of RUL,
2) a calibrated confidence band,
3) model and service state metadata,
4) validation or error state information if applicable.

The confidence band is used as a compact uncertainty communication mechanism. The objective is not to claim full probabilistic uncertainty quantification, but to provide a calibrated interval-like representation that improves interpretability for the downstream risk engine and dashboard. In methodological terms, this means that the output of the predictive layer is not just a scalar regression value, but a structured inference record that can be consumed safely by the rest of the architecture.

C. Deterministic agent layer

1) Design principle: The agent layer follows a deterministic-first design. This means that the core decision logic of the system does not rely on a large language model as the source of truth for risk or maintenance advice. Instead, the agent layer uses explicit validation, deterministic rules, and auditable transformations to map predictive outputs into decision-oriented artifacts.

This design is intentional for three reasons. First, it preserves traceability. Second, it reduces unpredictability in safety-relevant outputs. Third, it allows the decision process to remain inspectable even when optional language-model assistance is unavailable. Any language-model functionality is therefore auxiliary and non-authoritative.

2) Decision flow: The agent layer executes a structured orchestration pipeline:

1) Validate the incoming payload.
2) Adapt the payload to the predictive-layer contract.
3) Call the predictive layer and obtain RUL and confidence information.
4) Compute a deterministic `risk_score`.
5) Convert that score into a categorical `risk_level`.
6) Generate a `recommendation_text`.
7) Attach service-state and trace information.
8) Write an audit record.

The resulting output schema includes at least the following fields:

- `rul_pred`
- `confidence_band`
- `risk_score`
- `risk_level`
- `recommendation_text`
- `service_status`
- `trace`
- `audit_record_id`

The methodological significance of this layer lies in its role as a translation boundary. It converts predictive inference into maintenance semantics while preserving explicit state propagation and auditability.

3) Risk logic and recommendation construction: The risk engine transforms predictive outputs into decision variables through deterministic scoring logic. Although the exact implementation is rule-based and system-specific, the conceptual mapping is straightforward: lower estimated RUL and less favorable confidence conditions produce higher risk scores, which are then discretized into operationally interpretable maintenance states such as healthy, warning, or critical.

Recommendation generation is also deterministic. Rather than producing free-form advice from a generative model, the system assembles recommendation text from explicit logic tied to the inferred maintenance state. This ensures that the recommendation remains aligned with the actual predictive and risk outputs and can be traced back to structured inputs.

D. Scenario analysis

1) Purpose of what-if scenarios: Scenario analysis is included to transform the PHM system from a passive estimator into an exploratory decision-support tool. Instead of receiving only the current engine-state assessment, the user can test controlled perturbations to operating or sensor variables and inspect how the inferred RUL and risk outputs change under hypothetical conditions.

2) Scenario parsing and validation: Scenario input is designed to support compact expressions such as `cycle +25`, `sensor_11 -0.1`, or `set op_setting_1 to 0.6`. A deterministic parser is always applied first. If deterministic parsing fails and an optional language-model client is available, the system may attempt assisted interpretation; however, any language-model interpretation must still pass deterministic validation before execution.

This two-stage design is methodologically important. It preserves flexibility for natural-language-like scenario inputs while ensuring that all executed perturbations remain governed by explicit validation rules. Unsafe or invalid scenarios are therefore rejected with deterministic error handling rather than silently interpreted.

E. Dashboard layer as operationalization interface

Although the dashboard is presented to the user as the final interface, in methodological terms it is not merely a visualization layer. It is the operationalization surface of the full PHM stack. The dashboard consumes outputs from the predictive and agent layers and exposes them through four principal workflows:

1) **Summary:** compact system state and maintenance interpretation,
2) **Analysis:** expanded predictive and risk evidence,
3) **Scenarios:** baseline-versus-perturbed comparisons,
4) **Technical Audit:** trace records, model state, and service metadata.

The dashboard therefore acts as a multi-source consumer of predictive inference, deterministic decision logic, and audit infrastructure. Its role in the methodology is to ensure that all system outputs remain inspectable by both operational users and technically oriented reviewers.

F. Reproducibility and artifact design

Reproducibility is treated as a system-level design requirement rather than as a documentation afterthought. The proposed PHM stack is organized through explicit boundaries between predictive inference, deterministic decision logic, and dashboard-level operationalization. This separation is supported by versioned code artifacts, fixed preprocessing rules, champion and fallback configuration records, structured audit logs, and contract-driven interfaces between layers. As a result, the same input payload can be traced through predictive output, risk translation, recommendation logic, and dashboard presentation in a controlled and inspectable manner.

From an implementation perspective, reproducibility is supported by the preservation of trained-model artifacts, evaluation outputs, audit records, and configuration files associated with champion selection and degraded-state handling. The technical-audit view of the dashboard further exposes this structure by surfacing service status, model version, trace information, and audit identifiers. In this sense, reproducibility in the present work is not limited to rerunning code; it also includes the ability to inspect how a given maintenance-oriented output was produced from a defined set of system components and contracts.

G. Methodological interpretation

Taken together, the proposed methodology defines a PHM pipeline in which benchmark prediction is embedded inside a structured decision system. The predictive layer answers the question *how much useful life remains*; the agent layer answers *what that implies for risk and action*; and the dashboard layer answers *how that information is inspected, compared, and audited*. This layered formulation is the main methodological contribution of the work, because it redefines the C-MAPSS task from isolated regression into an end-to-end PHM workflow with explicit architectural boundaries and deployment-oriented behavior.

V. Experimental Setup

A. Benchmark configuration

The experimental study is conducted on the four standard NASA C-MAPSS subsets: FD001, FD002, FD003, and

FD004. These subsets differ in operating-condition complexity and fault-mode diversity, which makes them suitable not only for aggregate model comparison, but also for evaluating robustness under heterogeneous degradation regimes. In the present work, the benchmark is treated as the predictive foundation of the PHM system, and therefore experiments are designed to assess both model accuracy and deployment-oriented behavior.

The predictive layer is evaluated over all four subsets rather than on a single benchmark partition. This decision is important because a system-level PHM pipeline must behave consistently across simpler and more complex operating regimes. A model with a favorable global metric but unstable subset behavior may still be undesirable for deployment if such instability propagates into the downstream risk and recommendation logic.

B. Candidate models under evaluation

Four predictive candidates are evaluated:

- **Random Forest Regressor (RF)**
- **HistGradientBoosting Regressor (HGB)**
- **Long Short-Term Memory (LSTM)**
- **Gated Recurrent Unit (GRU)**

These models were chosen to span two distinct modeling families. RF and HGB represent row-based tabular baselines with relatively simple serving behavior, while LSTM and GRU represent sequence-aware neural models designed to exploit temporal degradation structure more directly. The purpose of the comparison is therefore not only to identify the numerically strongest predictor, but also to evaluate the trade-off between predictive performance and system integration readiness.

C. Evaluation dimensions

The experimental protocol is structured around three complementary dimensions:

1) **Predictive performance:** how accurately the model estimates RUL across all benchmark subsets.
2) **Engineering performance:** how efficiently and reliably the model can be served within the PHM architecture.
3) **System integration suitability:** how well the model supports deterministic downstream decision logic, fallback behavior, and operational traceability.

This three-part evaluation reflects the broader objective of the paper. Since the proposed work is not limited to a benchmark regression study, the selection of the deployed model must account for both predictive and operational criteria.

D. Predictive metrics

Predictive quality is assessed primarily through Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE). These metrics are reported both globally and by subset. Global metrics summarize overall predictive quality across the combined benchmark, while subset-specific metrics expose model behavior under different combinations of operating conditions and fault modes.

RMSE is used as the main comparative metric for headline model ranking because it penalizes larger errors more strongly and is widely used in RUL studies. MAE is reported as a complementary metric because it provides a more directly interpretable average absolute deviation in cycle units. Reporting both metrics makes it easier to distinguish whether a model's error profile is dominated by large outliers or by more uniform prediction deviations.

E. Engineering metrics

In addition to predictive metrics, the experimental setup includes engineering-oriented measurements. These are necessary because the predictive layer is part of a deployable PHM stack rather than an isolated modeling benchmark. The principal engineering metric reported in this study is inference latency, measured as total wall-clock time and average milliseconds per sample for each candidate model.

Latency is particularly relevant because the predictive model feeds a deterministic agent layer and an interactive dashboard interface. A model that offers marginal predictive gains at the cost of materially higher serving complexity may weaken the overall system even if it improves raw RMSE. Engineering evaluation therefore complements predictive ranking by asking whether a model can support reliable, stable, and practical inference within the deployed architecture.

F. Champion selection protocol

The champion-selection process is intentionally broader than "choose the lowest RMSE model." Instead, the deployed champion is selected through a mixed criterion that includes:

1) global predictive quality,
2) subset-level stability,
3) inference behavior and serving simplicity,
4) compatibility with deterministic downstream logic,
5) architectural clarity for fallback and degraded-state handling.

Under this protocol, the best raw benchmark model and the best deployed model are allowed to differ. This distinction is central to the paper. The model with the lowest aggregate predictive error is not automatically promoted to deployment if its integration or serving profile introduces unnecessary system-level fragility.

G. Fallback evaluation

A fallback design is explicitly included in the experimental setup. In addition to the deployed champion, a backup predictive model is retained so that the system can continue operating under controlled degraded states if the champion becomes unavailable or fails validation. The existence of this fallback is treated as part of the experimental design because it affects architectural resilience and downstream decision continuity.

Accordingly, the evaluation does not stop at identifying a single best model; it also considers whether the overall predictive layer can maintain usable behavior under failure or substitution conditions. This makes the experiment closer to real PHM deployment practice, where service continuity and controlled degradation matter alongside nominal accuracy.

TABLE III
GLOBAL MODEL METRICS.

Model	RMSE	MAE	Rows
GRU	17.424	13.194	25,309
RF	18.313	13.256	32,267
HGB	18.433	13.329	32,267
LSTM	20.012	14.990	25,309

H. Scenario and dashboard evidence

The experiments also include system-level evidence beyond pure prediction. Scenario analysis is used to compare baseline and perturbed inputs, allowing inspection of how hypothetical operating changes affect RUL and risk outputs. In parallel, dashboard evidence is collected to verify that the outputs of the predictive and agent layers can be consumed coherently through summary, analysis, scenario, and technical-audit views.

These elements are not treated as cosmetic add-ons. They are part of the evaluation because the contribution of the paper lies in demonstrating a full PHM workflow rather than only a prediction engine. As a result, the experimental setup is designed to connect model behavior, decision logic, and operational presentation within a single reproducible framework.

I. Experimental interpretation

In summary, the experimental setup is designed to answer two distinct but related questions. The first is predictive: which candidate model performs best on NASA C-MAPSS under global and subset-level metrics? The second is architectural: which model can serve most effectively as the predictive core of an end-to-end PHM system with deterministic decision logic, fallback support, and audit-oriented operationalization? The results section is structured accordingly, beginning with predictive comparisons and then moving toward engineering trade-offs, champion selection, scenario behavior, and dashboard evidence.

VI. RESULTS

A. Global predictive performance

Table III summarizes the global predictive metrics of the four candidate models. Among all evaluated models, GRU achieves the best raw aggregate performance, with an RMSE of 17.424 and an MAE of 13.194. Random Forest and HistGradientBoosting follow closely, with RF obtaining an RMSE of 18.313 and HGB an RMSE of 18.433. LSTM shows the weakest overall predictive performance in the current comparison, with an RMSE of 20.012 and an MAE of 14.990.

From a benchmark-only perspective, these results would naturally favor GRU as the strongest predictive model. However, the numerical gap between GRU and the best-performing tree-based models is relatively modest, especially when compared against the broader system-level requirements of the proposed PHM architecture. This observation becomes more important when the results are disaggregated by subset and then contrasted with engineering metrics.

TABLE IV
PER-DATASET RMSE BY MODEL.

Model	FD001	FD002	FD003	FD004
GRU	13.974	19.427	14.121	17.989
RF	18.686	19.600	16.978	17.512
HGB	18.680	19.763	17.065	17.654
LSTM	18.784	21.728	15.733	20.468

TABLE V
LATENCY AND ENGINEERING TRADE-OFFS.

Model	Samples	Total seconds	ms/sample
HGB	512	5.493	10.728
RF	512	0.161	0.313
LSTM	28,149	0.383	0.014
GRU	25,309	1.319	0.052

B. Subset-level behavior

Table IV reports subset-level RMSE across FD001–FD004. GRU leads on FD001, FD002, and FD003, while RF achieves the best RMSE on FD004. More importantly, the subset-level comparison shows that RF and HGB remain consistently competitive across all four benchmark partitions, without the large degradation that might make a model operationally fragile in a mixed-regime deployment setting.

Two observations follow from this table. First, sequence-aware models do provide an advantage in predictive sharpness, particularly in the simpler and more temporally structured subsets. Second, that advantage is not uniformly dominant across all regimes. The fact that RF performs best on FD004 and remains near the top across the other subsets supports the interpretation that tree-based baselines retain strong value in C-MAPSS, especially when system-level deployment criteria are taken into account.

C. Engineering trade-offs

Table V reports inference latency and average milliseconds per sample. These measurements reveal a key difference between predictive ranking and deployment suitability. HGB shows the highest latency among the tabular models, whereas RF offers very low latency at approximately 0.313 ms/sample. The recurrent models show low per-sample times in the reported batch-style evaluation, but their integration profile remains more complex because they depend on windowed sequence handling and a more involved serving path than the row-based baselines.

These results should not be interpreted only as raw speed rankings. In the present system, engineering suitability includes not just latency but also inference simplicity, serving stability, feature-contract clarity, and compatibility with deterministic downstream logic. From that broader perspective, RF provides an especially favorable trade-off: it remains highly competitive in predictive error while preserving a simpler and more stable deployment boundary.

TABLE VI
SCENARIO COMPARISON EXAMPLE.

Metric	Baseline	Scenario
RUL prediction	123.955	123.874
Risk level	healthy	healthy
Risk score	35	35
Service status	ok	ok
ΔRUL	-0.081	
ΔRisk score	0	

D. Champion selection outcome

The most important system-level result is that the best benchmark model is not the deployed champion. Although GRU achieves the best raw global RMSE, the selected champion for deployment is Random Forest, with HistGradientBoosting retained as fallback. This decision reflects the mixed evaluation protocol defined in Section V, where model selection is driven by both predictive and engineering criteria.

The rationale is straightforward. GRU is the strongest predictor in the narrow benchmark sense, but RF provides stronger integration readiness, more stable serving behavior, simpler inference contracts, and a cleaner interface with the deterministic agent layer. In a paper focused exclusively on predictive modeling, this decision might appear conservative. In an end-to-end PHM system, however, it is methodologically coherent: the deployed model must support the full decision workflow, not only minimize aggregate error.

This result is one of the central findings of the paper. It demonstrates that model selection for PHM deployment should be framed as a system-design problem rather than as a leaderboard-only optimization exercise.

E. Scenario evidence

To evaluate the system beyond nominal prediction, scenario analysis was used to compare baseline and perturbed engine states. Table VI presents a representative example. In the reported case, the baseline and scenario predictions are nearly identical: the baseline RUL is 123.955 cycles, the scenario RUL is 123.874 cycles, and the resulting change is $\Delta\text{RUL} = -0.081$ cycles. The risk level remains *healthy*, and the risk score remains unchanged at 35.

This result is useful for two reasons. First, it shows that the scenario mechanism is functioning end to end, from input perturbation through predictive inference and deterministic risk translation. Second, it illustrates that not all domain-meaningful perturbations produce large predictive consequences. In other words, the scenario engine is not artificially amplifying changes; it preserves local model sensitivity and therefore supports a more realistic interpretation of hypothetical interventions.

F. Failure handling and contract evidence

A deployment-oriented PHM system must also behave predictably under invalid, degraded, or unavailable states. Table VII summarizes representative failure modes and their handling strategies. Invalid request payloads produce deterministic validation errors, predictive validation failures propagate explicit service states, champion unavailability activates the fallback model, optional language-model failures do not break the deterministic path, and unsafe scenarios are rejected through validation logic.

TABLE VII
FAILURE MODES AND HANDLING STRATEGY.

Failure mode	Handling strategy
Invalid request payload	Deterministic validation response with explicit error message
Predictive validation failure	Propagate `error_validacion` service state
Champion unavailable	Use deterministic fallback model and state propagation
LLM key/provider failure	Continue with rules-only deterministic path
Unsafe scenario perturbation	Reject scenario with validation errors and safety notes

TABLE VIII
CONTRACT AND TRACE FIELD MAPPING.

Dashboard tab	Field	Source layer
Summary	`rul_pred`, `risk_level`	agent / predictive
Analysis	`risk_score`, `confidence_band`, `trace`	agent / predictive
Scenarios	`comparison`, `change_summary`	agent scenario
Technical Audit	`audit_record_id`, `model_version`, `service_status`	agent / predictive

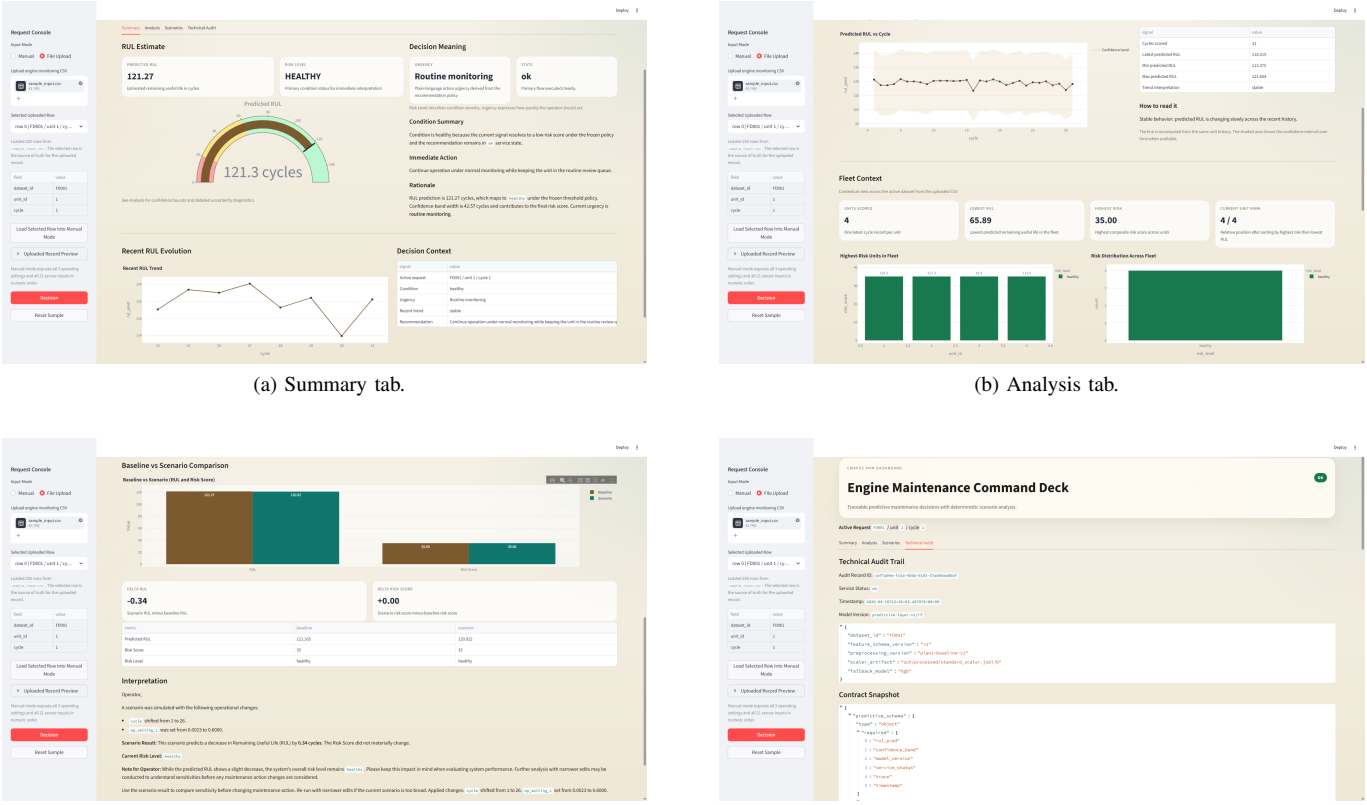
Table VIII complements this view by showing how dashboard-visible fields map back to their source layers. This contract-driven structure is important because it allows technical review of where each visible decision artifact originates within the architecture.

Together, these results show that the system is not only producing predictions, but also maintaining traceable and controlled behavior across normal and degraded execution paths.

G. Dashboard evidence

Figure 2 shows the four principal dashboard views: Summary, Analysis, Scenarios, and Technical Audit. These views demonstrate that the layered architecture can be operationalized coherently for both user-facing interpretation and technical inspection. The Summary tab presents a compact maintenance state; the Analysis tab exposes predictive and risk evidence; the Scenarios tab supports baseline-versus-perturbed comparison; and the Technical Audit tab surfaces structured records and service metadata.

At the level of individual scenario visualization, Figure 3 separates physical prediction and decision-oriented interpretation. The left panel compares baseline and scenario RUL estimates, while the right panel shows the scenario risk score. This dual representation is methodologically important because it



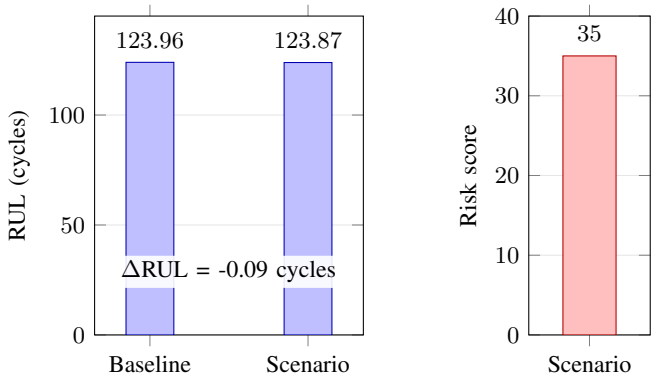
(a) Summary tab.

(b) Analysis tab.

(c) Scenarios tab.

(d) Technical Audit tab.

Fig. 2. Dashboard evidence across operator and audit workflows. The four panels show how the PHM system is operationalized through summary-level maintenance interpretation, detailed predictive and risk analysis, scenario comparison, and audit-oriented technical trace inspection.



(a) Baseline-versus-scenario RUL comparison.

(b) Scenario risk score.

Fig. 3. Scenario comparison split into physical prediction and decision-oriented risk: (a) baseline and scenario RUL estimates with the associated ΔRUL, and (b) the scenario risk score.

makes clear that the system does not collapse all reasoning into a single scalar output. Instead, it preserves the distinction between predictive state and maintenance interpretation.

H. Result synthesis

Taken together, the results support three main conclusions. First, GRU is the strongest model in raw predictive terms, but RF remains close enough in benchmark performance to be a serious deployment candidate. Second, once engineering trade-offs, fallback design, and deterministic integration are considered, RF becomes the more suitable champion for the full PHM system. Third, the scenario, contract, and dashboard results show that the contribution of the work is not restricted to RUL modeling: it is the construction of a traceable end-to-end PHM workflow in which predictions are translated into inspectable maintenance intelligence.

VII. Discussion

A. Benchmark winner versus deployment champion

The central finding of this work is that the best predictive model in benchmark terms is not necessarily the best model for deployment inside an end-to-end PHM system. In the present results, GRU achieves the best raw global RMSE, which would make it the natural winner under a benchmark-only perspective. However, the deployed champion is Random Forest, not because GRU is weak, but because the design objective of the system is broader than minimizing aggregate prediction error.

This distinction is methodologically important. A deployed PHM component must operate within a larger architecture that includes validation logic, deterministic risk translation, fallback behavior, scenario analysis, and audit-oriented inspection. In that context, model quality is not only a function of RMSE or MAE, but also of serving stability, interface clarity, failure handling, and compatibility with downstream logic. Random Forest performs well enough on the benchmark to remain highly credible as a predictor, while also offering a cleaner operational boundary than the sequence-based alternatives. The result is a model that may be slightly less sharp in raw predictive terms, but more suitable as the core of a reproducible and inspectable PHM workflow.

This is not an argument against sequence models. Rather, it is an argument against reducing system design to leaderboard optimization. Sequence-aware architectures remain valuable and may become the best deployment candidates under different latency, interpretability, or infrastructure assumptions. What the present work shows is that model selection in PHM deployment must be framed as a systems problem rather than a purely predictive contest.

B. Why deterministic-first design matters

A second major implication concerns the role of deterministic logic in engineering AI systems. In the proposed architecture, the predictive layer estimates RUL and communicates calibrated confidence information, but it does not directly produce the final maintenance decision. That role belongs to the deterministic agent layer, which computes risk scores, assigns maintenance states, generates recommendation text, and propagates service-state information through explicit rules.

This separation adds value in at least three ways. First, it improves traceability. Because the decision logic is deterministic, the system can expose why a particular recommendation was produced and how it relates to predictive outputs. Second, it improves operational stability. The maintenance recommendation does not depend on the latent variability of a generative model, but on inspectable mappings from structured inputs to structured outputs. Third, it improves failure tolerance. Even if optional language-model assistance becomes unavailable, the PHM workflow can continue through its deterministic path without losing core functionality.

From a research perspective, this deterministic-first design is one of the strongest features of the system. It converts the predictive model from an isolated estimator into one component of a controlled decision pipeline. This matters especially in engineering contexts, where the trustworthiness of the system depends not only on numerical accuracy, but also on whether the system remains understandable and reviewable when something goes wrong.

C. Interpreting scenario sensitivity

The scenario results also deserve careful interpretation. In the representative case shown in the results section, the scenario perturbation produces only a very small change in predicted RUL and no change in the resulting risk category. At first glance, one might interpret this as evidence that the scenario system is weak or uninformative. However, the opposite interpretation is more appropriate.

A realistic scenario-analysis mechanism should preserve the local sensitivity structure of the underlying predictive model and the deterministic thresholds of the risk logic. This means that not every perturbation should lead to a large change in outcome. If the user changes an input that is excluded from the deployed feature subset, or changes a retained feature in a region of relatively low local sensitivity, then a small response is not a defect but a faithful reflection of the model and system design. In other words, the scenario module is not intended to dramatize hypothetical changes; it is intended to expose how the current predictive and decision pipeline reacts to them.

This observation also highlights a limitation of scenario reasoning in benchmark-derived PHM systems. Scenario effects are only as meaningful as the underlying feature set, model sensitivity, and risk thresholds allow. Therefore, the value of what-if exploration depends not only on interface design, but also on how well the predictive layer captures physically relevant state changes and how carefully the downstream rules are calibrated.

D. Prediction is only the first layer of PHM usefulness

Another important conclusion is that point prediction alone is not an adequate endpoint for PHM system design. The C-MAPSS literature has traditionally emphasized the quality of the estimated RUL value, which is a valid and necessary objective. However, a scalar prediction becomes practically useful only after it is contextualized. In the proposed system, contextualization happens through calibrated confidence bands, deterministic risk translation, maintenance recommendations, scenario comparison, and technical audit records.

This layered treatment changes the role of the predictive model. Instead of being the final product, it becomes an intermediate analytical component whose outputs must support downstream maintenance intelligence. This architectural shift is one of the main reasons why the dashboard and audit views matter. They are not merely presentation devices; they are the means through which the system demonstrates that predictions can be inspected, interpreted, compared, and traced.

For this reason, the contribution of the present work should not be read as "Random Forest is better than GRU." That would be too narrow and would miss the actual contribution. The stronger conclusion is that an end-to-end PHM system must be evaluated at multiple levels simultaneously: predictive accuracy, engineering operability, decision-layer coherence, and audit-oriented transparency.

E. Physics-informed extensions as a future direction

A meaningful future extension of the present system is the incorporation of physics-informed learning. Physics-informed neural networks (PINNs) and, more broadly, physics-informed machine learning aim to combine data with governing equations, constraints, or structured prior knowledge in order to improve generalization and physical consistency [12], [13]. In

a related perspective, theory-guided data science argues that scientific machine learning systems benefit when known domain structure is embedded directly into the learning process rather than treated as an external interpretation layer [14].

For the C-MAPSS setting, this does not imply that a full PDE-level engine model can be inserted directly into the current benchmark pipeline. The benchmark provides operational settings and sensor trajectories rather than a complete set of governing equations for internal flow and thermodynamic fields. Nevertheless, physics-informed extensions could still be valuable in at least three ways. First, loss functions could be augmented with degradation-consistency constraints or monotonicity priors tied to known health progression patterns. Second, surrogate physical structure could be injected into latent representations so that predictions remain more consistent across operating regimes and fault modes. Third, hybrid architectures could be explored in which data-driven predictors are regularized by simplified engine-health relations or operator-learning components inspired by recent neural-operator work [15].

In this sense, PINNs should be viewed here not as an immediate replacement for the current predictive layer, but as a promising future direction for strengthening physical plausibility and regime robustness. Such a direction would be especially relevant if future versions of the project incorporate richer simulator access, additional health-state variables, or higher-fidelity degradation constraints than those available in the present benchmark-only setup.

F. What this paper contributes beyond a benchmark study

Viewed as a whole, the paper contributes a change in emphasis for C-MAPSS-based research. The benchmark is preserved, the predictive problem is preserved, and the model comparison remains meaningful. But the final artifact is no longer only a benchmark experiment. It is a deployable PHM prototype with explicit contracts between layers, controlled fallback behavior, scenario reasoning, and traceable outputs.

This distinction matters because it moves the research closer to how PHM systems are actually consumed in engineering settings. Real systems are not judged only by whether one model reduces RMSE by a fraction of a cycle. They are judged by whether predictions can be turned into reliable action, whether degraded behavior remains controlled, and whether a reviewer can later inspect how a maintenance state was produced. In that sense, the present work argues for a broader evaluation culture in PHM: one in which benchmark excellence is important, but system coherence is equally central.

VIII. Limitations

The present work has several important limitations that should be stated explicitly.

First, the system is built on top of NASA C-MAPSS, which is a simulated benchmark rather than a real operational fleet dataset. Although C-MAPSS is highly valuable for controlled comparison and reproducible experimentation, it cannot fully capture the variability, sensing imperfections, maintenance history, and organizational constraints of real deployment environments. As a result, the predictive and system-level conclusions of this paper should be interpreted as benchmark-grounded evidence rather than field validation.

Second, the deployed predictive pipeline uses a fixed feature subset and a specific preprocessing policy. This improves interface stability and reproducibility, but it also constrains what the system can perceive. In particular, scenario analysis is limited by the sensitivity of the selected features and by the information content available in the retained variables. Consequently, some plausible perturbations may lead to only minor changes in RUL or risk outputs, not because the scenario framework is defective, but because the predictive layer is only responsive to the information encoded in the deployed feature contract.

Third, the risk engine and recommendation layer are deterministic and rule-based. This is a deliberate design choice that improves traceability, inspectability, and failure tolerance, but it also means that the resulting maintenance semantics are heuristic rather than empirically validated against real maintenance policy outcomes. The system therefore demonstrates a coherent and auditable decision pathway, but not a fully validated maintenance optimization policy.

Fourth, the scenario mechanism is constrained by parser design, validation rules, and the predictive model's local response surface. Although this makes the system safer and more controlled, it also limits the expressiveness of hypothetical exploration. Scenario outputs should therefore be interpreted as controlled perturbation analyses within the operational envelope of the deployed pipeline, not as fully general counterfactual reasoning.

Fifth, the dashboard layer should be understood as a deployment-oriented prototype rather than a production-certified interface. It successfully demonstrates end-to-end operationalization, including summary, analysis, scenario, and technical-audit views, but it has not been evaluated under real user studies, real-time service constraints, or long-term production monitoring conditions.

Finally, the champion-selection conclusion is specific to the present architectural context. The fact that Random Forest is preferred over GRU in this system should not be generalized into a universal claim about PHM modeling. Under different infrastructure assumptions, different interpretability requirements, or different sequence-serving constraints, the preferred deployment model could change. The contribution of this paper is therefore not a universal ranking of model families, but a demonstration that deployment-oriented PHM model selection must be made at the system level rather than from benchmark error alone.

IX. Conclusion

This paper presented an end-to-end PHM system built on the NASA C-MAPSS benchmark, with the explicit objective of moving beyond standalone Remaining Useful Life (RUL) prediction toward a deployment-oriented maintenance intelligence workflow. The proposed architecture integrates three

separated but connected layers: a predictive layer for calibrated RUL estimation, a deterministic agent layer for risk scoring and recommendation generation, and a dashboard layer for operational interpretation, scenario exploration, and technical auditability.

From the predictive standpoint, the study showed that strong performance can be achieved with both sequence-based and tree-based models, but that raw benchmark accuracy alone is insufficient for deployment-oriented model selection. Although GRU achieved the best raw global RMSE in the evaluated portfolio, Random Forest was selected as the deployed champion because it provided a better balance between predictive competitiveness, inference simplicity, serving stability, fallback compatibility, and integration clarity within the full PHM stack. This result reinforces a central argument of the paper: in practical PHM systems, the best benchmark model is not automatically the best deployed model.

From the system-design standpoint, the main contribution of the work is the transformation of a benchmark-oriented prognostics task into a traceable and inspectable PHM artifact. The proposed system demonstrates that RUL prediction becomes significantly more useful when embedded within deterministic risk logic, scenario comparison, and audit-oriented operational views. In this architecture, prediction is not the final endpoint, but the first stage of a broader maintenance decision workflow.

More broadly, this work argues for a shift in how C-MAPSS-based PHM studies are framed. Benchmark comparisons remain important, but they should be complemented by evaluation of engineering operability, controlled degraded-state behavior, and transparency of decision pathways. In that sense, the paper contributes not only a comparative modeling result, but also a reproducible systems perspective on how benchmark prognostics can be converted into deployment-oriented PHM applications.

Future work should extend the present prototype in three directions. First, the predictive layer can be strengthened through richer uncertainty treatment, improved sequence-serving strategies, and broader sensitivity analysis. Second, the decision layer can be refined through better-calibrated maintenance policies and more systematic validation of recommendation rules. Third, the system should be tested under more realistic operational settings, including user-centered evaluation, service monitoring, and adaptation to data closer to real fleet environments. These directions would further bridge the gap between benchmark excellence and practical PHM deployment.

Author Contributions

The author conceived the overall project, designed the end-to-end PHM architecture, implemented the predictive, agent, and dashboard layers, conducted the model comparison and champion-selection analysis, developed the deterministic risk and scenario logic, generated the experimental evidence and visual artifacts, and wrote and revised the manuscript in full.

Acknowledgments

The author acknowledges the Artificial Intelligence and Advanced Large Models course at Beihang University for providing the academic context in which this project was developed.

References

[1] D. K. Frederick, J. A. DeCastro, and J. S. Litt, "User's guide for the commercial modular aero-propulsion system simulation (c-mapss)," NASA and U.S. Army Research Laboratory, Tech. Rep. NASA/TM-2007-215026, 2007.

[2] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *Proceedings of the International Conference on Prognostics and Health Management (PHM)*, 2008.

[3] S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, "Long short-term memory network for remaining useful life estimation," in *IEEE International Conference on Prognostics and Health Management*, 2017.

[4] E. Ramasso and A. Saxena, "Prognostics in condition-based maintenance: From statistical and machine learning approaches to uncertainty management," in *European Conference of the Prognostics and Health Management Society*, 2014.

[5] J. Z. Sikorska, M. Hodkiewicz, and L. Ma, "Prognostic modelling options for remaining useful life estimation by industry," *Mechanical Systems and Signal Processing*, vol. 25, no. 5, pp. 1803–1836, 2011.

[6] J. Lee, H.-A. Kao, and S. Yang, "Service innovation and smart analytics for industry 4.0 and big data environment," *Procedia CIRP*, vol. 16, pp. 3–8, 2014.

[7] X.-S. Si, W. Wang, C.-H. Hu, and D.-H. Zhou, "Remaining useful life estimation — a review on the statistical data-driven approaches," *European Journal of Operational Research*, vol. 213, no. 1, pp. 1–14, 2011.

[8] F. O. Heimes, "Recurrent neural networks for remaining useful life estimation," in *2008 International Conference on Prognostics and Health Management*, 2008.

[9] G. S. Babu, P. Zhao, and X.-L. Li, "Deep convolutional neural network based regression approach for estimation of remaining useful life," in *Database Systems for Advanced Applications*. Springer, 2016, pp. 214–228.

[10] A. L. Ellefsen, E. Bjørlykhaug, V. Æsøy, S. Ushakov, and H. Zhang, "Remaining useful life predictions for turbofan engine degradation using semi-supervised deep architecture," *Reliability Engineering & System Safety*, vol. 183, pp. 240–251, 2019.

[11] A. Saxena, J. Celaya, E. Balaban, K. Goebel, B. Saha, S. Saha, and M. Schwabacher, "Metrics for evaluating performance of prognostic techniques," in *2008 International Conference on Prognostics and Health Management*, 2008.

[12] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[13] G. E. Karniadakis, I. G. Kevrekidis, and L. Yang, "Physics-informed machine learning," *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.

[14] A. Karpatne, G. Atluri, J. H. Faghmous, M. Steinbach, A. Banerjee, A. Ganguly, S. Shekhar, N. F. Samatova, and V. Kumar, "Theory-guided data science: A new paradigm for scientific discovery from data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 29, no. 10, pp. 2318–2331, 2017.

[15] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," in *International Conference on Learning Representations (ICLR)*, 2021.